# WareHouseApp – Inventory Management System

CS107.3 Object Oriented Programming with C# (Repeat) – Coursework Report

## 1. Problem Statement

Small and medium warehouses often track stock, customers and staff on paper or in scattered spreadsheets. This causes several recurring problems: stock-outs that are noticed too late because nobody is alerted when an item runs low; duplicated or inconsistent customer records that waste time and damage service; and no control over who may view or change sensitive data such as employee salaries. Staff spend time searching for information, and managers cannot trust the figures they are given.

WareHouseApp is a Windows Forms desktop application, written in C# on .NET Framework 4.7.2, that keeps inventory, customers and employees in a single SQL Server database behind a login screen. It contributes to a solution in four ways. First, input validation stops bad data entering the system. Second, low-stock alerts on the home dashboard show exactly what must be reordered. Third, role-based access protects employee records from users who do not need them. Fourth, searchable records replace manual look-ups. Together these give staff one reliable source of truth, reduce errors and save time in daily warehouse work.

## 2. Requirements

The application is aimed at warehouse administrators, who need full control of stock, customer and staff records, and at shipping operators, who handle day-to-day stock and customer work but should not see employee records. Each feature below was chosen because it directly addresses a problem described above.

| Feature | Why it is included |
|---|---|
| Sign up & login | Only authorised staff may use the system. Passwords are stored as salted PBKDF2 hashes, never in plain text. A default administrator account is created on first run. |
| Inventory management | Add, edit, delete and search items with quantity, unit price, category, storage location and reorder level. Items at or below their reorder level are flagged on the dashboard. |
| Customer management | Maintain customer contact and company details with phone and email validation. |
| Employee management | Maintain staff records (NIC, position, salary, hire date). Restricted to the Admin role. |

Non-functional requirements: every feature is stored in the database; all user input is validated; all failures are caught and shown as clear messages instead of crashes; the system is interactive using mouse and keyboard (for example Enter submits the login form).

## 3. Design

System architecture. The application uses a layered architecture so the user interface is independent of data access:

| Layer | Components |
|---|---|
| User interface | Form1 (login), SignUpForm, DashBoard, MainDash, InventoryControl, CustomerControl, EmployeeControl |
| Services | AuthService – login, sign-up and password rules |
| Repositories | MaterialRepository, CustomerRepository, EmployeeRepository, UserRepository – parameterised SQL |
| Database | Database class (ADO.NET). Creates the WarehouseDB database and the tables Users, Materials, Customers and Employees on first run |

Data and code structure (class model).

| Class | Extends / implements | Key members |
|---|---|---|
| Entity (abstract) | – | Id, Validate() |
| Contact (abstract) | Entity | Name, Phone, Email, Address |
| Customer | Contact | Company |
| Employee | Contact | NIC, Position, Salary, HireDate |
| Material | Entity | Name, Category, Quantity (rule: never negative), UnitPrice, ReorderLevel, Location, IsLowStock |
| Person (abstract) | Entity | UserName, FullName, Email, Role, CanManageEmployees, Create() factory |
| Admin | Person | Role = Admin; CanManageEmployees = true |
| ShippingOperator | Person | Role = Operator; CanManageEmployees = false |
| IRepository<T> | interface | Search, Add, Update, Delete, Count |
| Material/Customer/EmployeeRepository | IRepository<T> | Parameterised SQL for each entity |
| ManagementControl (abstract) | UserControl | Grid, search box, Add/Update/Delete/Clear buttons, central error handling |
| Inventory/Customer/EmployeeControl | ManagementControl | Entity specific fields and mapping |
| ValidationException, AuthenticationException, DataAccessException | Exception | Custom exceptions for user mistakes, bad logins and database faults |

How the processes interact. Program initialises the database and then loops Login → Dashboard until the user exits. On a management screen the form fields are copied into an entity object, the entity validates itself, the repository runs a parameterised SQL command through the Database class, and any exception is translated into a friendly message by ManagementControl. Logging out returns to the login screen.

OOP concepts used: abstraction (Entity, Person, ManagementControl), inheritance (Contact → Customer/Employee, Person → Admin/ShippingOperator), polymorphism (Role and CanManageEmployees overrides, IRepository<T>, the template method in ManagementControl), encapsulation (Material.Quantity setter, private connection handling), a factory method (Person.Create) and custom exception types.

Why these structures are appropriate. The template-method base class removed duplicated screen code, so a new module needs only its own fields and queries. The repository interface keeps SQL out of the user interface and allows the data source to be changed or mocked. Parameterised queries prevent SQL injection, and role logic lives in the model classes rather than being scattered through the forms.

## 4. Database Design

| Table | Columns |
|---|---|
| Users | UserId (PK), Username (unique), PasswordHash, Salt, FullName, Email, Role |
| Materials | MaterialId (PK), Name, Category, Quantity, UnitPrice, ReorderLevel, Location, LastUpdated |
| Customers | CustomerId (PK), Name, Phone, Email, Address, Company |
| Employees | EmployeeId (PK), Name, NIC (unique), Phone, Email, Address, Position, Salary, HireDate |

The database and tables are created automatically by code the first time the application runs, so no manual database setup or attached .mdf file is required.

## 5. Implementation and Version Control

The work started from the partially developed starter project supplied with the assignment (login form, dashboard layout and a few stub classes). The starter was a Git repository containing the initial project files, the dashboard design work (branch Designing, merged into master through a pull request) and an early database commit. Development then followed these steps: (1) replace the hard-coded login with database authentication and password hashing; (2) add the Sign Up form; (3) build the shared ManagementControl base class; (4) implement Inventory, Customer and Employee modules on top of it; (5) wire the dashboard navigation, role-based access and statistics; (6) test and fix build issues, including restoring image resources the starter referenced but which were missing.

Version control approach: the project is kept in Git and hosted on GitHub at https://github.com/kariyawasamp679-create/WareHouseApp (branch main). The history begins with the starter project's commits and continues with one commit per stage of my work: (1) database layer, entity models, repositories and password hashing; (2) database-backed login and sign-up; (3) inventory, customer and employee management screens with dashboard navigation; (4) project file update; (5) project report; (6) removal of the unused Form2. Committing stage by stage kept each change small and reversible, and because the database schema is created from code, it is version-controlled together with the source.

## 6. Personal Reflection

Several things worked well. The abstract ManagementControl base class and the repository interface kept the three management screens consistent and made validation and error handling a one-place job, which saved a lot of duplicated code. Letting the application create its own database on first run removed manual setup steps and the dependency on an attached .mdf file. Hashing passwords and using parameterised queries from the start avoided security rework later. Testing the data layer directly against LocalDB found real problems early, such as a layout that was cropped by Windows font scaling.

Some things did not work well. The starter project referenced image files that were missing, which stopped it compiling, and its hard-coded login had to be replaced. A stale LocalDB registry entry on my machine

blocked the default instance until I added a fallback instance. Building some layouts in code rather than the designer was quick to write but harder to preview. In my next project I will write automated unit tests for the validation rules earlier, commit smaller changes more often, agree the database schema before building screens, and test on a clean machine sooner.

## 7. Future Work

The application could be developed further in several directions: stock movement history so every goods-in and goods-out event is recorded and auditable; customer orders and shipping status so inventory is reduced automatically when an order is dispatched; printable and exportable reports, for example stock valuation and reorder lists; barcode scanning to speed up stock counts; a change-password and user-management screen for administrators; and a language option, since the starter already contains a Language button.

To make it a more sustainable product I would move the connection string into a configuration file so it can change per environment, introduce a data-access interface so the database can be swapped or mocked in tests, add automated unit tests and logging, schedule regular database backups, and publish an installer so updates are easy to deploy. Keeping the layered structure and documenting each module would also make it easier for another developer to maintain.

## 8. Testing

The database layer and all three modules were tested against SQL Server 2022 LocalDB. Checks that passed: database and tables created on first start; admin login succeeds and a wrong password is rejected with AuthenticationException; sign-up creates an Operator who can then log in; inventory, customer and employee records can be added, searched, updated and deleted; invalid input is rejected with ValidationException (empty name, bad phone or email, missing NIC, negative salary or quantity, future hire date); a duplicate NIC is rejected by the database unique constraint; the Employees menu is disabled for the Operator role; and items at or below their reorder level appear in the low-stock alerts.

## 9. Screenshots

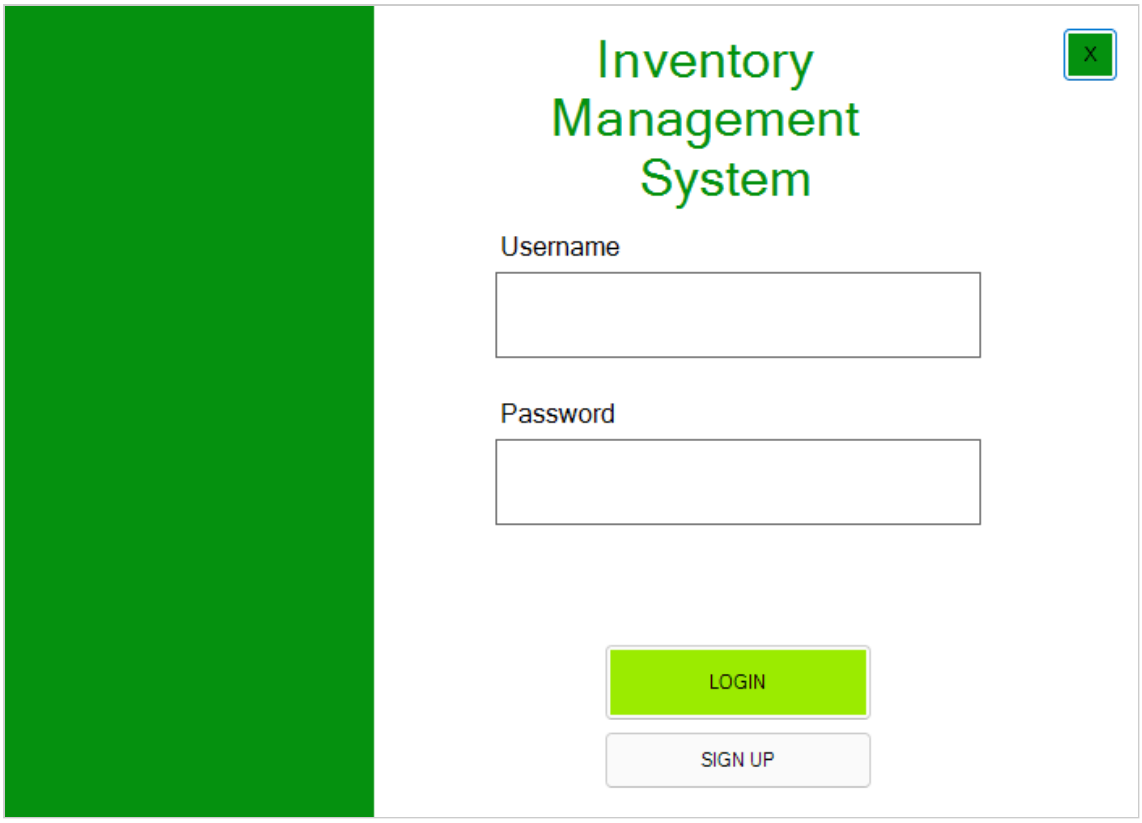



*Figure 1 – Login screen*

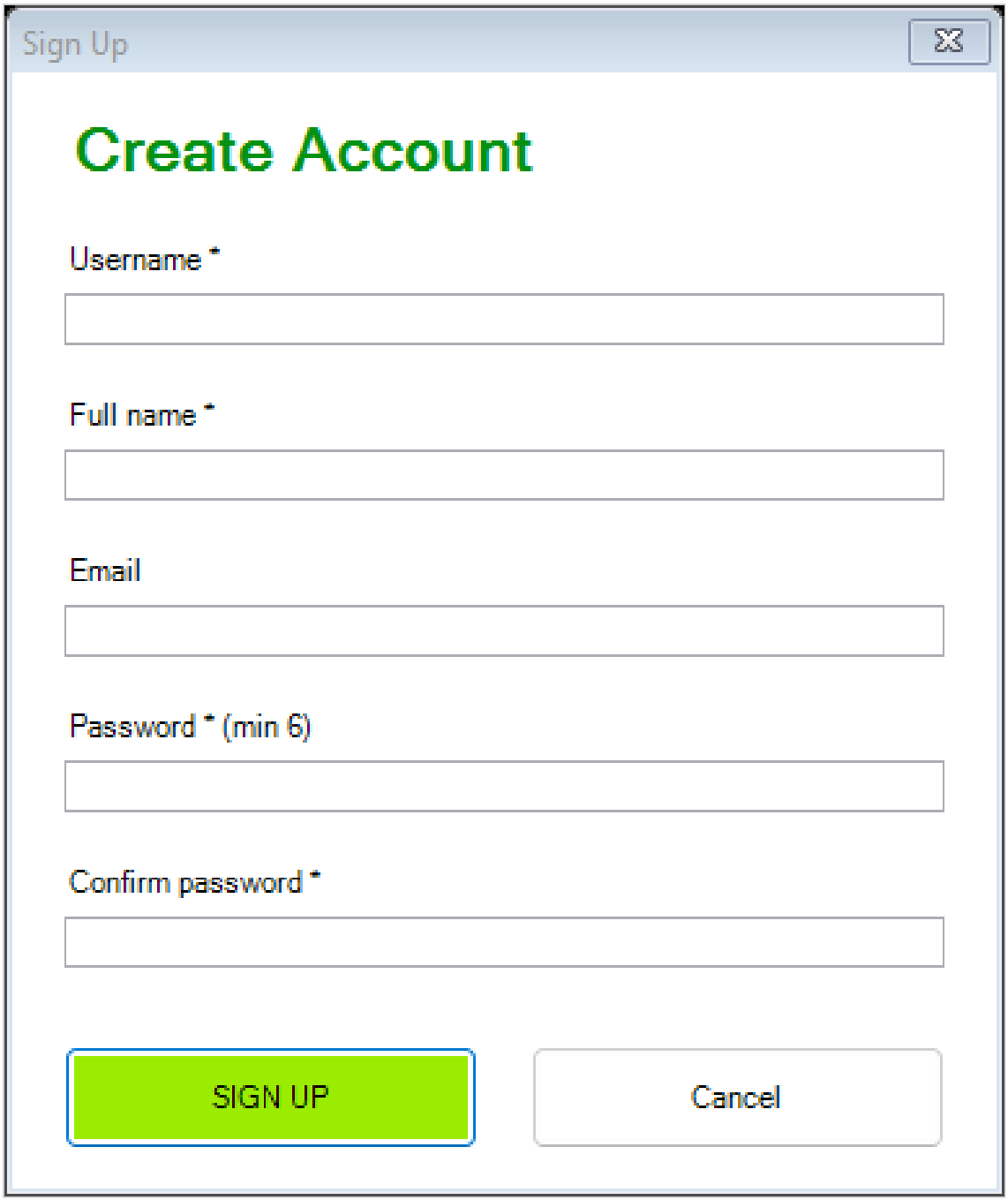



*Figure 2 – Sign Up dialog*

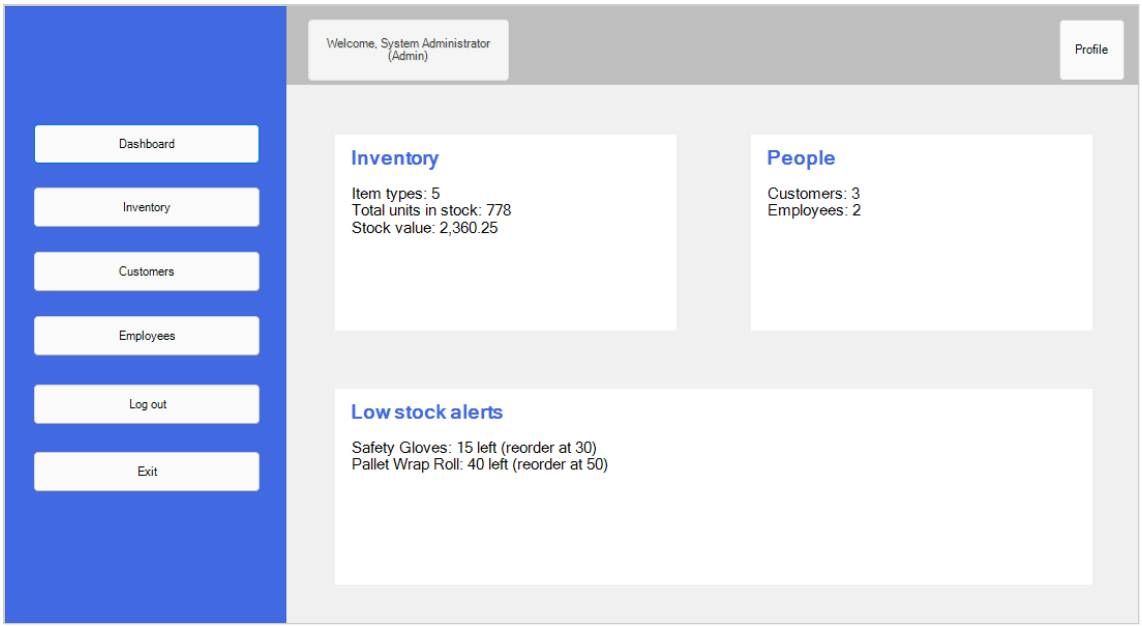



*Figure 3 – Dashboard with live statistics and low-stock alerts*

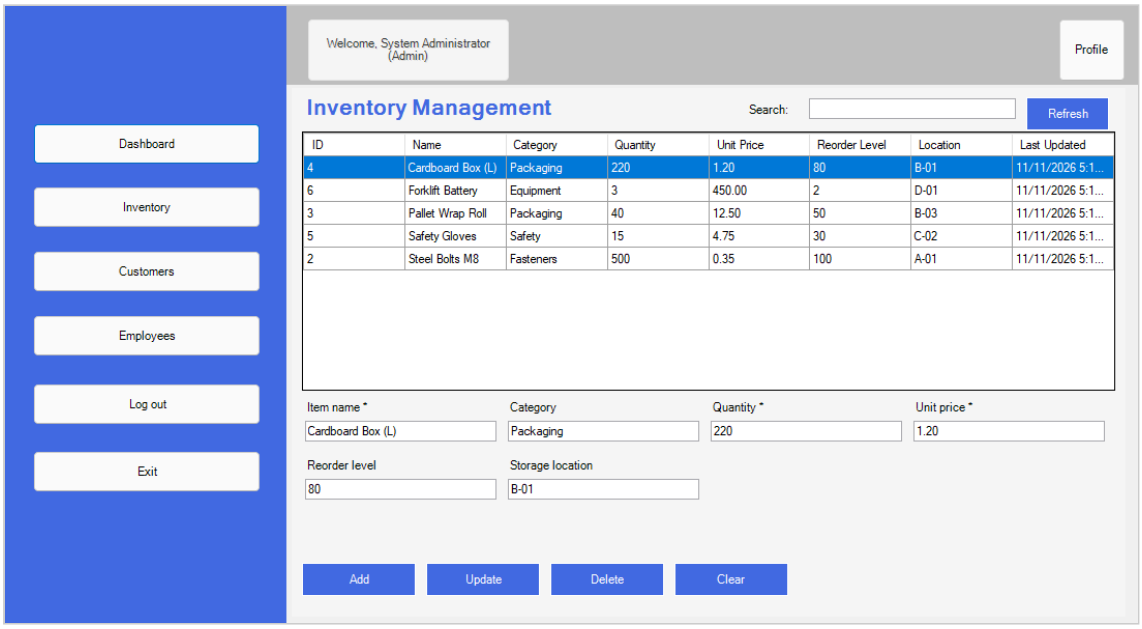



*Figure 4 – Inventory management*

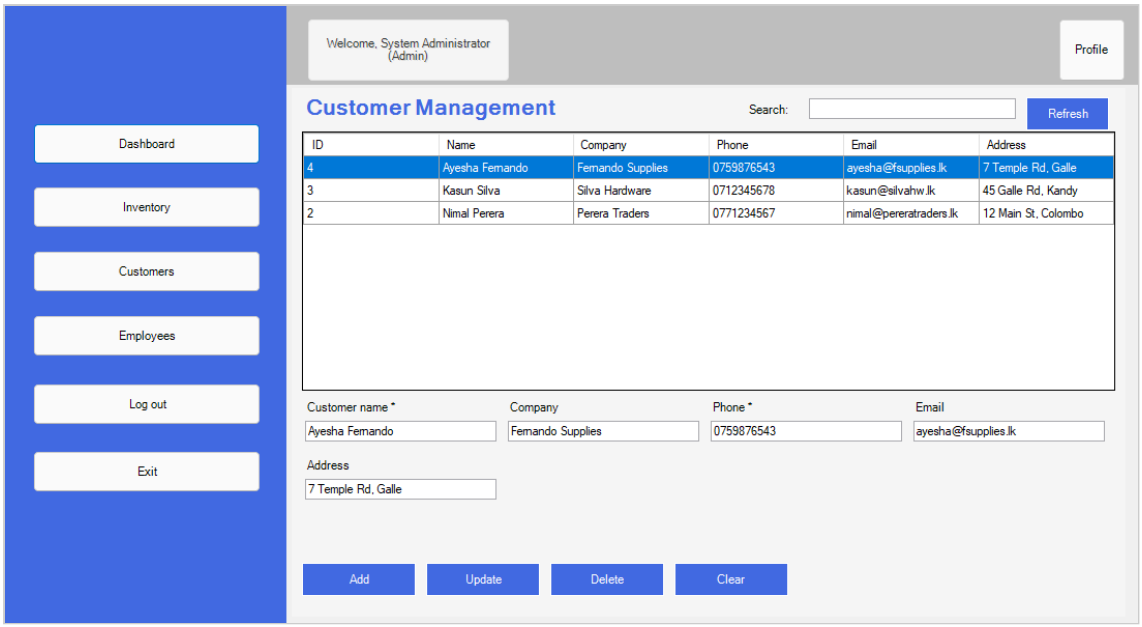



*Figure 5 – Customer management*

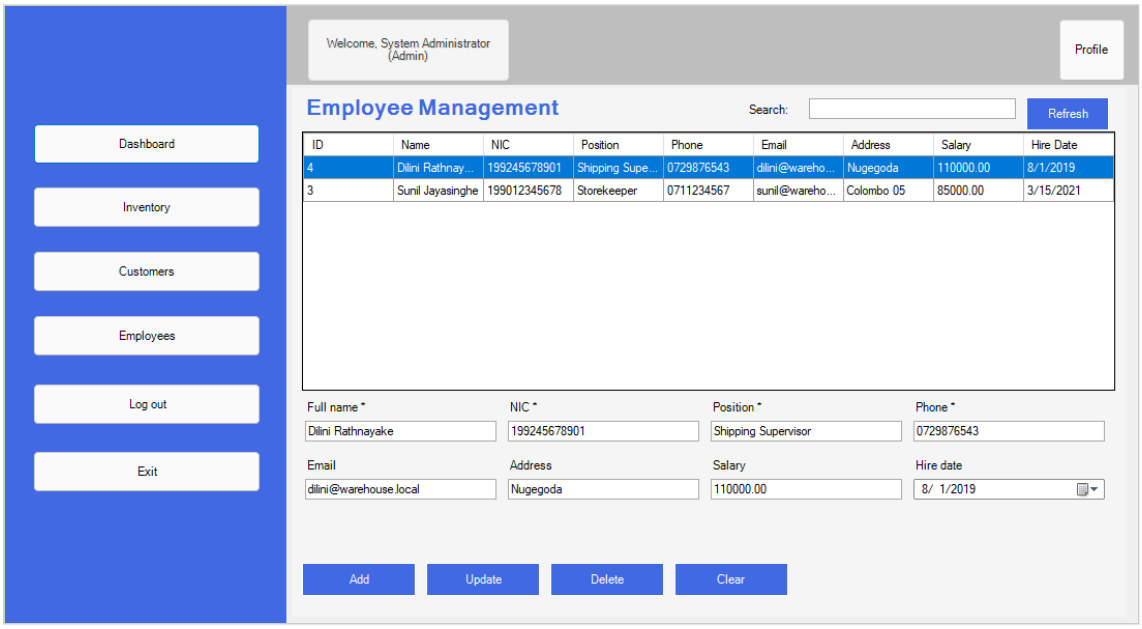



*Figure 6 – Employee management (Admin only)*

## 10. Video Demonstration

[Add unlisted YouTube link here]